

Zookeeper讲义

授课目标

- 目标01-了解Zookeeper主要应用场景
- 目标02-掌握Zookeeper搭建和配置
- 目标03-了解Zookeeper基本概念【节点】和数据存储模型
- 目标04-掌握Cli命令和Java代码操作Zookeeper服务
- 目标05-理解Zookeeper的监听机制
- 目标06-掌握Zookeeper集群搭建
- 目标07-理解案例：手写分布式锁

1. Zookeeper简介

1.1 什么是Zookeeper?

ZooKeeper为协调、构建分布式应用的高性能服务

ZooKeeper从字面意思理解Zoo-Keeper动物园-管理员。动物园中有很多动物，这里的动物就可以比作分布式环境下多种多样的服务，而ZooKeeper做的就是管理这些服务。

ZooKeeper是一个分布式的，开放源码的分布式应用程序协调服务！管理分布式系统中服务的一个服务！

ZooKeeper 本质上是一个分布式的小文件存储系统。提供基于类似于文件系统的目录树方式的数据存储，并且可以对树中的节点进行有效管理。从而用来维护和监控你存储的数据的状态变化。通过监控这些数据状态的变化，从而达到基于数据的集群管理。

ZooKeeper 适用于存储和协同相关的关键数据，不适合用于大数据量存储。

官方地址：<https://zookeeper.apache.org/releases.html>

应用场景：

- 配置中心
- 注册中心
- 分布式锁
- 分布式队列
- 负载均衡器
- DNS服务
- Master选举

1.2 发展历程

Zookeeper 最早起源于雅虎研究院的一个研究小组。当时研究人员发现，在雅虎内部很多大型系统基本都需要依赖一个系统来进行分布式协同，但是这些系统往往都存在分布式单点问题。所以，雅虎的开发人员就开发了一个通用的无单点问题的分布式应用程序的高性能协调服务，这就是Zookeeper。

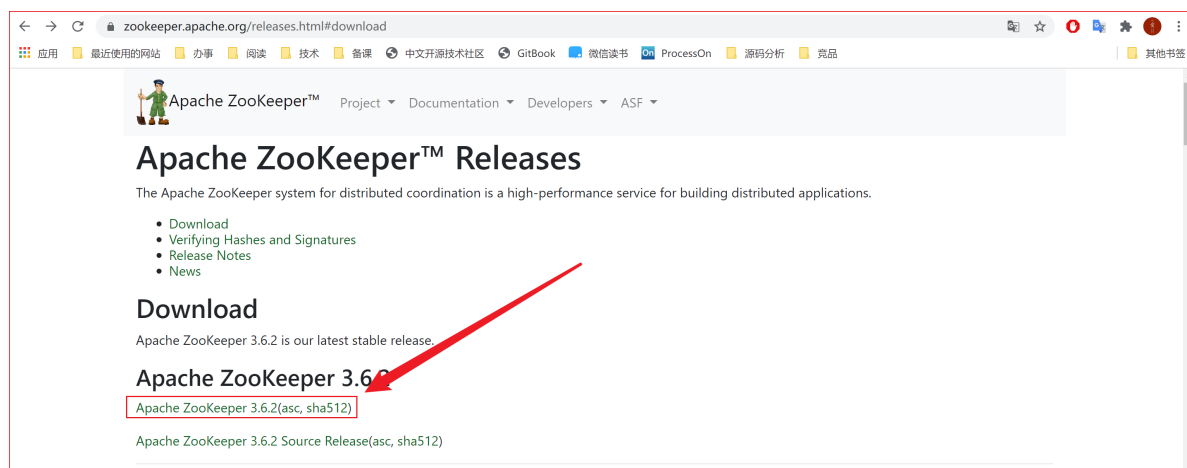
Zookeeper之后在开源界被大量使用，很多著名开源项目都在使用Zookeeper，例如：

- Hadoop：使用Zookeeper 做Namenode的高可用。
- HBase：保证集群中只有一个master，保存hbase:meta表的位置，保存集群中的RegionServer列表。
- Kafka：集群成员管理controller节点的选举。

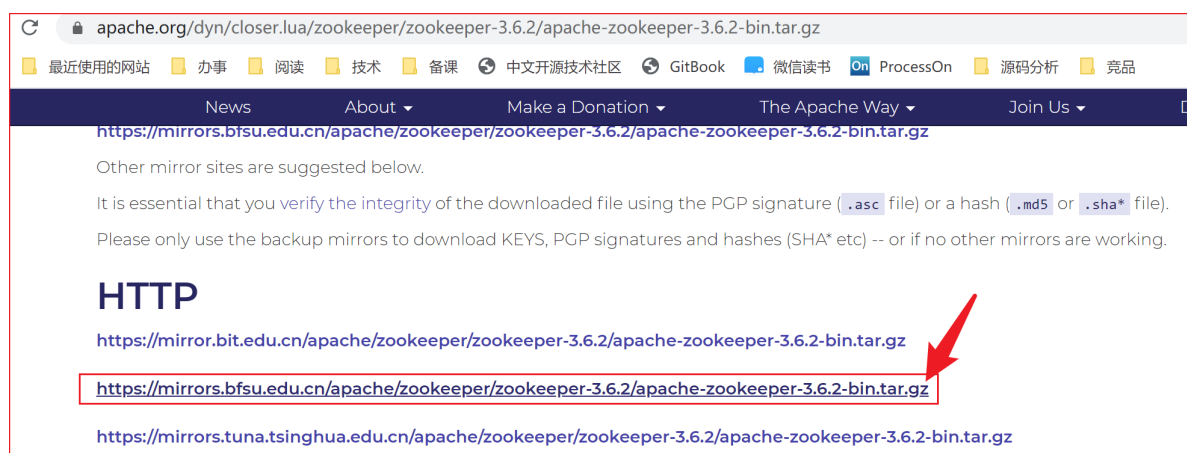
1.3 环境搭建

1.3.1 下载

下载地址：<http://zookeeper.apache.org>

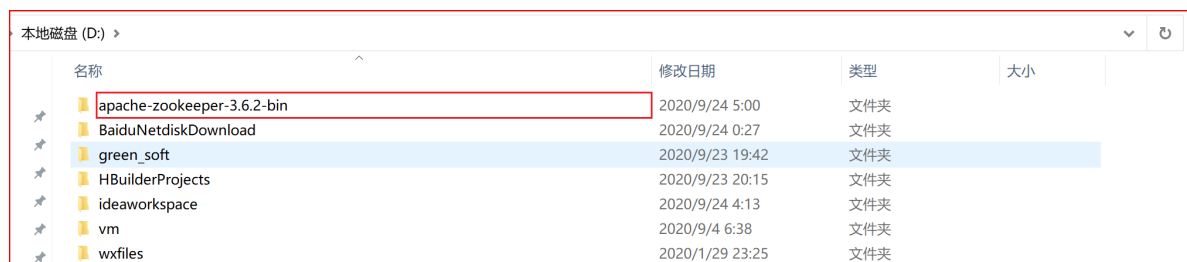


Zookeeper下载页的地址



1.3.2 解压

注意：解压到没有中文路径的目录下



1.3.3 修改配置文件

在Zookeeper路径下创建一个data目录

本地磁盘 (D:) > apache-zookeeper-3.6.2-bin >

名称	修改日期	类型	大小
bin	2020/9/24 4:59	文件夹	
conf	2020/9/24 4:59	文件夹	
data	2020/9/24 5:00	文件夹	
docs	2020/9/24 4:59	文件夹	
lib	2020/9/24 4:59	文件夹	
logs	2020/9/24 5:18	文件夹	
LICENSE.txt	2020/9/4 20:43	TXT 文件	12 KB
NOTICE.txt	2020/9/4 20:43	TXT 文件	1 KB
README.md	2020/9/4 20:43	Markdown File	2 KB
README_packaging.md	2020/9/4 20:43	Markdown File	4 KB

修改配置文件

conf路径中复制一份zoo_sample.cfg, 改名为 zoo.cfg

本地磁盘 (D:) > apache-zookeeper-3.6.2-bin > conf

名称	修改日期	类型	大小
configuration.xml	2020/9/4 20:43	XSL 样式表	1 KB
log4j.properties	2020/9/4 20:43	PROPERTIES 文件	4 KB
zoo.cfg	2020/9/24 4:59	CFG 文件	2 KB
zoo_sample.cfg	2020/9/4 20:43	CFG 文件	2 KB

指定保存数据的目录: data目录

D:\apache-zookeeper-3.5.8-bin\conf\zoo.cfg - Notepad++

```
1 # The number of milliseconds of each tick
2 tickTime=2000
3 # The number of ticks that the initial
4 # synchronization phase can take
5 initLimit=10
6 # The number of ticks that can pass between
7 # sending a request and getting an acknowledgement
8 syncLimit=5
9 # the directory where the snapshot is stored.
10 # do not use /tmp for storage, /tmp here is just
11 # example sakes.
12 dataDir=../data
13 # the port at which the clients will connect
14 clientPort=2181
15 # the maximum number of client connections.
```

1.3.4 启动Zookeeper

打开bin路径, 双击zkServer.cmd, 启动Zookeeper服务

本地磁盘 (D:) > apache-zookeeper-3.6.2-bin > bin

名称	修改日期	类型	大小
README.txt	2020/9/4 20:43	TXT 文件	1 KB
zkCleanup.sh	2020/9/4 20:43	Shell Script	3 KB
zkCli.cmd	2020/9/4 20:43	Windows 命令脚本	2 KB
zkCli.sh	2020/9/4 20:43	Shell Script	2 KB
zkEnv.cmd	2020/9/4 20:43	Windows 命令脚本	2 KB
zkEnv.sh	2020/9/4 20:43	Shell Script	4 KB
zkServer.cmd	2020/9/4 20:43	Windows 命令脚本	2 KB
zkServer.sh	2020/9/4 20:43	Shell Script	11 KB
zkServer-initialize.sh	2020/9/4 20:43	Shell Script	5 KB
zkSnapshotToolkit.cmd	2020/9/4 20:43	Windows 命令脚本	1 KB
zkSnapshotToolkit.sh	2020/9/4 20:43	Shell Script	2 KB
zkTxnLogToolkit.cmd	2020/9/4 20:43	Windows 命令脚本	1 KB
zkTxnLogToolkit.sh	2020/9/4 20:43	Shell Script	2 KB

1.3.5 启动客户端测试

启动客户端，看到 'Welcome to Zookeeper!' 说明成功

```
C:\WINDOWS\system32\cmd.exe
. 0.202\bin;C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common;C:\Program Files (x86)\MySQL\MySQL Server 5.5\bin;F:\va
apache-maven-3.3.9\bin;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsPowerShell\v1.0
\;C:\WINDOWS\System32\OpenSSH\;E:\tools\qq\Bin;C:\Program Files\Git\cmd;C:\Program Files\TortoiseGit\bin;C:\Users\sun\AppData\Local\Microsoft\WindowsApps;C:\Program Files (x86)\SSH Communications Security\SSH Secure Shell;.
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:java.io.tmpdir=C:\Users\sun\AppData\
Local\Temp\
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:java.compiler=<NA>
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:os.name=Windows 10
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:os.arch=amd64
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:os.version=10.0
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:user.name=sun
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:user.home=C:\Users\sun
2019-11-04 09:24:23,032 [myid:] - INFO [main:Environment@100] - Client environment:user.dir=F:\zookeeper-3.4.13\bin
2019-11-04 09:24:23,032 [myid:] - INFO [main:ZooKeeper@442] - Initiating client connection, connectString=localhost:2181
l_sessionTimeout=30000 watcher=org.apache.zookeeper.ZooKeeperMain$MyWatcher@69d0a921
Welcome to ZooKeeper!
2019-11-04 09:24:23,345 [myid:] - INFO [main-SendThread(localhost:2181):ClientCnxn$SendThread@1029] - Opening socket co
nnection to server localhost/127.0.0.1:2181. Will not attempt to authenticate using SASL (unknown error)
2019-11-04 09:24:23,345 [myid:] - INFO [main-SendThread(localhost:2181):ClientCnxn$SendThread@879] - Socket connection
established to localhost/127.0.0.1:2181, initiating session
JLine support is enabled
[zk: localhost:2181(CONNECTING) 0] 2019-11-04 09:24:23,360 [myid:] - INFO [main-SendThread(localhost:2181):ClientCnxn$S
endThread@1303] - Session establishment complete on server localhost/127.0.0.1:2181, sessionId = 0x1000a668ca20000, nego
tiated timeout = 30000
WATCHER::
WatchedEvent state:SyncConnected type:None path:null
[zk: localhost:2181(CONNECTED) 0]
```

2. 基本操作

2.1 数据存储模型

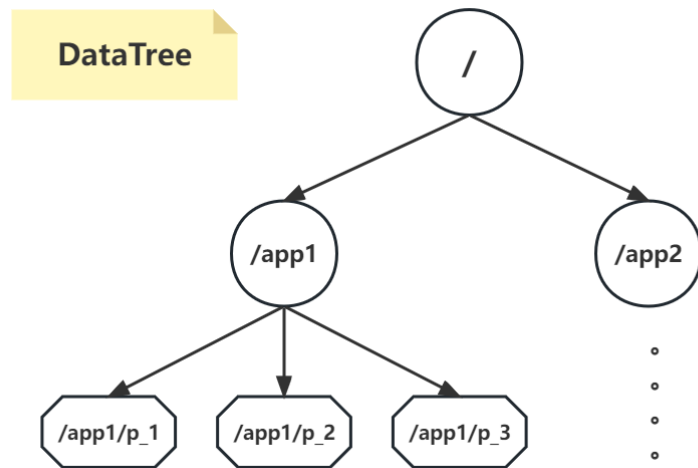
Zookeeper 数据模型是层次模型【树结构】。层次模型常见于文件系统。

例如：我的电脑可以分为多个盘符（例如C、D、E等），每个盘符下可以创建多个目录，每个目录下面可以创建文件，也可以创建子目录，最终构成了一个树型结构。通过这种树型结构的目录，我们可以将文件分门别类的进行存放，方便我们后期查找。而且磁盘上的每个文件都有一个唯一的访问路径，例如：C:\Windows\hero\hello.txt。

层次模型和key-value 模型是两种主流的数据模型。Zookeeper 使用文件系统模型主要基于以下两点考虑：

- 便于表达数据之间的层次关系。
- 便于为不同的应用分配独立的命名空间（namespace）。

Zookeeper 的层次模型称作DataTree。DataTree 的每个节点叫作**znode**。不同于文件系统，每个节点都可以保存数据。每个节点都有一个版本(version)，版本从0 开始计数。



如图所示，DataTree中有两个子树，用于应用1(/app1)和应用2 (/app2) 。每个客户端进程创建一个znode节点 p_i 在 /app1下， /app1/p₁就代表一个客户端在运行。

2.2 节点分类

znode可以分为持久性节点和临时性节点：

- 持久性persistent：持久化到磁盘上。持久化节点一经创建不会丢失，无论是Server宕机还是Client宕机。
- 临时性ephemeral：保存在内存中，没有保存到硬盘。如果Server宕机或者client在指定的超时时间内没有连接Server都会丢失。

znode也可以是带排序号有序的节点，顺序性节点关联一个唯一的递增整数，这个整数是节点名字后缀

- 持久顺序性persistent_sequential：持久性 + 顺序性
- 临时顺序性ephemeral_sequential：临时性 + 顺序性

2.3 客户端命令

2.2.1 查询所有命令 help

```
[zk: localhost:2181(CONNECTED) 2] help
Zookeeper -server host:port -client-configuration properties-file cmd args
addWatch [-m mode] path # optional mode is one of [PERSISTENT, PERSISTENT_RECURSIVE] - default is PERSISTENT_R
ECURSIVE
addauth scheme auth
close
config [-c] [-w] [-s]
connect host:port
create [-s] [-e] [-c] [-t ttl] path [data] [acl]
delete [-v version] path
deleteall path [-b batch size]
delquota [-n|-b] path
get [-s] [-w] path
getAcl [-s] path
getAllChildrenNumber path
getEphemerals path
history
listquota path
ls [-s] [-w] [-R] path
printwatches on/off
```

2.2.2 ls查询根路径下的节点

```
[zk: localhost:2181(CONNECTED) 1] ls / 查询根路径下的节点
[dubbo, zookeeper]
[zk: localhost:2181(CONNECTED) 2] ls /zookeeper 查询zookeeper节点下的节点
[quota]
```

2.2.3 创建节点

1) 创建普通节点app1

```
1  ## create 节点名称 节点对应value值
2  ## 参数含义
3  ## -s 生成序号
4  ## -e 临时存储
5  create /app1 'hello hero'
```

2) 创建普通节点app2

sequential 顺序的

```
1  create -s /app2 'app2Node'
```

```
[zk: localhost:2181(CONNECTED) 0] create -s /hello 'hello' 创建带有需要的永久性节点 hello, 会自动添加序号
Created /hello000000000006
[zk: localhost:2181(CONNECTED) 1] create -s /hello 'hello'
Created /hello000000000007
[zk: localhost:2181(CONNECTED) 2] ls /
[dubbo, zookeeper, app1, hello000000000006, hello000000000007] 执行两次命令, 每次创建的节点序号不同
[zk: localhost:2181(CONNECTED) 4] create /hello 'hello'
Created /hello
[zk: localhost:2181(CONNECTED) 5] create -s /hello/a 'a' 以下是在hello节点中创建两个带有序号的a节点
Created /hello/a000000000000
[zk: localhost:2181(CONNECTED) 6] create -s /hello/a 'aa'
Created /hello/a000000000001
[zk: localhost:2181(CONNECTED) 7] ls /hello
[a00000000000, a000000000001]
[zk: localhost:2181(CONNECTED) 8]
```

3) 创建普通临时节点app3 create -e

ephemeral 短暂的;瞬息的

```
1  create -e /app3 'app3'
```

```
[zk: localhost:2181(CONNECTED) 8] create -e /app3 'app3' 创建临时的节点
Created /app3
```

关闭客户端, 再次打开查看 app3节点消失

注意: 再次打开客户端并不会立即消失, 需要等待10秒左右, 如果想要立即消失, 退出客户端采用quit命令。

4) 创建带序号临时节点app4 create -e -s

```
1  create -e -s /app4 'app4'
```

```
[zk: localhost:2181(CONNECTED) 1] create -e -s /app4 'app4'
Created /app400000000011
```

关闭客户端, 再次打开查看 app4节点消失

2.2.4 查看节点

1) 查询节点数据 get

```
1 ## get 节点名称
2 get /app1
```

```
[zk: localhost:2181(CONNECTED) 1] get /app1
helloworld
```

2) 查看节点状态 stat

```
1 ## get 节点名称
2 stat /app1
```

```
[zk: localhost:2181(CONNECTED) 2] stat /app1
cZxid = 0x2
ctime = Sun Jan 15 15:02:54 CST 2023
mZxid = 0x2
mtime = Sun Jan 15 15:02:54 CST 2023
pZxid = 0x2
cversion = 0
dataVersion = 0
aclVersion = 0
ephemeralOwner = 0x0
dataLength = 10
numChildren = 0
[zk: localhost:2181(CONNECTED) 3] █
```

```
1 ## -----节点的状态信息，也称为stat结构体-----
2 cZxid = 0x17f ## 该数据节点被创建时的事务id
3 #其中zxid表示的是zookeeper的事务ID，由64位数字组成，分为高32位和低32位
4 ctime = Sat Dec 21 19:47:36 CST 2019 ## 该数据节点创建时间
5 mZxid = 0x17f ## 该数据节点被修改时最新的事物id
6 mtime = Sat Dec 21 19:47:36 CST 2019 ## 该数据节点最后更新时间
7 pZxid = 0x183 ## 当前节点的父级节点事务ID
8 cversion = 4 ## znode子节点变化号，znode子节点修改次数
9 dataVersion = 0 ## znode数据变化号
10 aclVersion = 0 ## 访问控制列表的变化号 access control
    list
11 ephemeralOwner = 0x0 ## 如果临时节点，表示当前节点的拥有者的
    sessionId。如果不是临时节点，则值为0
12 dataLength = 6 ## 数据长度
13 numChildren = 4 ## 子节点数据
```

2.2.5 修改和删除

1) 修改节点数据 set

```
1 ## 修改节点
2 set /app1 'app1Node'
```

```
[zk: localhost:2181(CONNECTED) 9] get /app1
[zk: localhost:2181(CONNECTED) 10] set /app1 'app1Node'
[zk: localhost:2181(CONNECTED) 11] get /app1
app1Node
```

2) 删除一个节点 delete

```
1 ## 删除节点
2 delete /hello0000000006
```

```
[zk: localhost:2181(CONNECTED) 14] ls /
[zookeeper, dubbo, hello, appl, hello0000000006, hello0000000007]
[zk: localhost:2181(CONNECTED) 15] delete /hello0000000006
[zk: localhost:2181(CONNECTED) 16] ls /
[zookeeper, dubbo, hello, appl, hello0000000007]
```

3) 删除多层多个节点 deleteall

```
1 ## 删除多层多个节点
2 deleteall /app3
3 ## 注意：低版本中没有这个命令
```

```
[zk: localhost:2181(CONNECTED) 30] ls /appl      查询/appl下的节点
[node1, node2]
[zk: localhost:2181(CONNECTED) 31] delete /appl   使用delete命令删除/appl节点, 提示节点非空
Node not empty: /appl
[zk: localhost:2181(CONNECTED) 32] deleteall /appl 使用deleteall命令删除/appl节点, 再次查询, /appl节点消失
[zk: localhost:2181(CONNECTED) 33] ls /
[config, dubbo, hello, zookeeper]
```

2.4 案例：使用代码操作ZK

ZooKeeper常用Java客户端:

1. 原生API: Zookeeper 原生Java API位于org.apache.Zookeeper包中
2. ZkClient: Github上一个开源的Zookeeper客户端, 由datameer的工程师Stefan Groschup和Peter Voss一起开发。zkclient-x.x.Jar也是在原生 api 基础之上进行扩展的开源 Java 客户端
3. Apache Curator: Apache Curator是 Zookeeper的Java客户端库
 - Curator项目的目标是简化Zookeeper客户端的使用
 - Curator最初是Netflix研发后来捐献给Apache基金会目前是Apache顶级项目

1) 创建项目导入依赖

项目名称zookeeper-example

导入依赖

```
1 <properties>
2   <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
3   <java.version>1.8</java.version>
4   <maven.compiler.source>1.8</maven.compiler.source>
5   <maven.compiler.target>1.8</maven.compiler.target>
6 </properties>
7
8 <dependencies>
9   <dependency>
10    <groupId>junit</groupId>
11    <artifactId>junit</artifactId>
12    <version>4.11</version>
```



```

13     </dependency>
14     <dependency>
15         <groupId>org.projectlombok</groupId>
16         <artifactId>lombok</artifactId>
17         <version>1.16.10</version>
18     </dependency>
19
20     <dependency>
21         <groupId>org.apache.zookeeper</groupId>
22         <artifactId>zookeeper</artifactId>
23         <version>3.4.8</version>
24     </dependency>
25
26     <dependency>
27         <groupId>org.apache.curator</groupId>
28         <artifactId>curator-client</artifactId>
29         <version>4.0.0</version>
30         <exclusions>
31             <exclusion>
32                 <groupId>org.apache.zookeeper</groupId>
33                 <artifactId>zookeeper</artifactId>
34             </exclusion>
35         </exclusions>
36     </dependency>
37     <dependency>
38         <groupId>org.apache.curator</groupId>
39         <artifactId>curator-framework</artifactId>
40         <version>4.0.0</version>
41         <exclusions>
42             <exclusion>
43                 <groupId>org.apache.zookeeper</groupId>
44                 <artifactId>zookeeper</artifactId>
45             </exclusion>
46         </exclusions>
47     </dependency>
48     <dependency>
49         <groupId>org.apache.curator</groupId>
50         <artifactId>curator-recipes</artifactId>
51         <version>4.0.0</version>
52     </dependency>
53 </dependencies>

```

2) 创建节点

```

1 public class Demo01CRUD {
2     //创建节点
3     @Test
4     public void demoHelloWorld() throws Exception {
5         //1.创建Zookeeper的客户端对象，配置重试策略
6         //参数1 : baseSleepTimeMs 两个重试之间休眠时间
7         //参数2 : maxRetries 最大重试次数
8         RetryPolicy retryPolicy = new ExponentialBackoffRetry(3000, 3);
9         //参数1: connectString服务ip地址:端口
10        //参数2: retryPolicy重试策略
11        //参数3: sessionTimeoutMs 会话超时时间

```

```

12         //参数4: connectionTimeoutMs 连接超时时间
13         CuratorFramework client =
CuratorFrameworkFactory.newClient("127.0.0.1:2181", retryPolicy);
14         //2.开启客户端
15         client.start();
16         //3.创建节点对象
17         client.create().forPath("/app2");
18         //4.关闭客户端，释放资源
19         client.close();
20     }
21     CuratorFramework client;
22
23     @Before
24     public void initClient() {
25         //初始化Zookeeper的客户端对象client
26         String connectString = "127.0.0.1:2181";
27         RetryPolicy retryPolicy = new ExponentialBackoffRetry(3000, 100);
28         int sessionTimeoutMs = 60 * 1000; //当前客户端会话超时时间
29         int connectionTimeoutMs = 15 * 1000; //连接超时时间
30         client = CuratorFrameworkFactory
31             .newClient(connectString, sessionTimeoutMs,
connectionTimeoutMs, retryPolicy);
32     }
33     @After
34     public void destroy(){
35         client.close();
36     }
37
38     @Test
39     public void createNode() throws Exception {
40         //3种方式，四种节点类型
41         //1.开启客户端
42         client.start();
43         //2.创建节点对象
44         //方式1: 创建空节点
45         //client.create().forPath("/app2");
46         //方式2: 创建有内容节点
47         //client.create().forPath("/app3", "app3Node".getBytes());
48         //方式3: 创建多层节点
49
50         //client.create().creatingParentsIfNeeded().forPath("/app4/a", "aa".getBytes
());
51         //节点节点类型1 : 持久节点CreateMode.PERSISTENT
52         //节点节点类型2 : 临时节点【客户端关闭则节点消失】CreateMode.EPHEMERAL
53
54         //client.create().withMode(CreateMode.EPHEMERAL).forPath("/app5", "app5Node"
.getBytes());
55         //节点节点类型3 : 持久节点+自带序号 CreateMode.PERSISTENT_SEQUENTIAL
56         //client.create().withMode(CreateMode.PERSISTENT_SEQUENTIAL).forPath("/app6
", "app6Node".getBytes());
57         //节点节点类型4 : 临时节点+自带序号【客户端关闭则节点消失】
CreateMode.EPHEMERAL_SEQUENTIAL
58         //client.create().forPath("/app5", "app5Node".getBytes());

```

```

57         //client.create().withMode(CreateMode.EPHEMERAL_SEQUENTIAL).forPath("/app6"
58         ,"app6Node".getBytes());
59
60         //Thread.sleep(9000); //休眠9秒，观察效果
61         //3.关闭客户端，释放资源
62         client.close();
63     }
64 }

```

3) 修改节点数据

```

1     @Test
2     public void updateNode() throws Exception {
3         //1.开启客户端
4         client.start();
5         //2.修改节点
6         client.setData().forPath("/app1", "app1Node".getBytes());
7         //3.关闭客户端，释放资源
8         client.close();
9     }

```

4) 节点数据查询

```

1     @Test
2     public void getNode() throws Exception {
3         //1.开启客户端
4         client.start();
5         //2.获取节点数据
6         byte[] nodeDataBytes = client.getData().forPath("/app1");
7         System.out.println("节点数据 = " + new String(nodeDataBytes));
8         //3.关闭客户端，释放资源
9         client.close();
10    }

```

5) 删除节点

```

1     @Test
2     public void deleteNode() throws Exception {
3         //1.开启客户端
4         client.start();
5         //2.获取节点数据
6         //方式1：删除一个节点
7         //client.delete().forPath("/app3");
8         //方式2：递归删除多个节点
9         //client.delete().deletingChildrenIfNeeded().forPath("/app3");
10        //方式3：强制删除【避免一些因为网络传输导致的删除不成功】
11        client.delete().guaranteed().forPath("/app2");
12        //3.关闭客户端，释放资源
13        client.close();
14    }

```

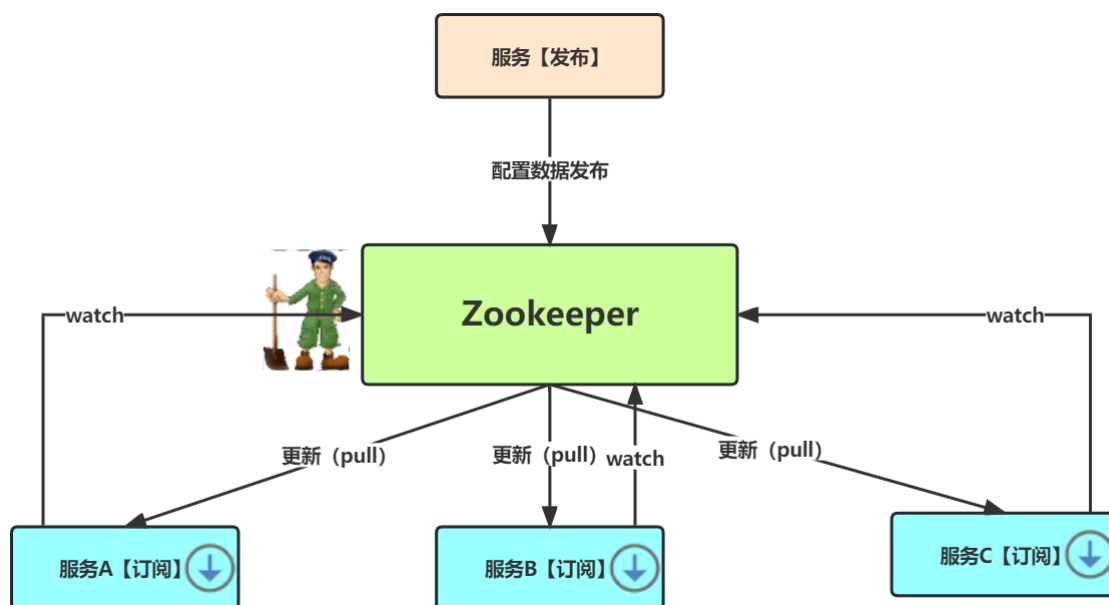
3. ZK监听机制详解

3.1 watch机制如何运作？

Zookeeper的watch机制本质就是订阅发布！采用了推拉结合的模式。

- **推**：服务端感知到内容变化了，会发送事件信息给关注watch节点的客户端。事件本身是轻量的不含变更内容，这是“推”。
- **拉**：客户端收到变更通知事件，客户端需要自己去拉变更数据，这是“拉”。

发布订阅上是观察者模式的一种实现。观察者订阅对应主题节点，节点一旦变化会自动通知观察者。



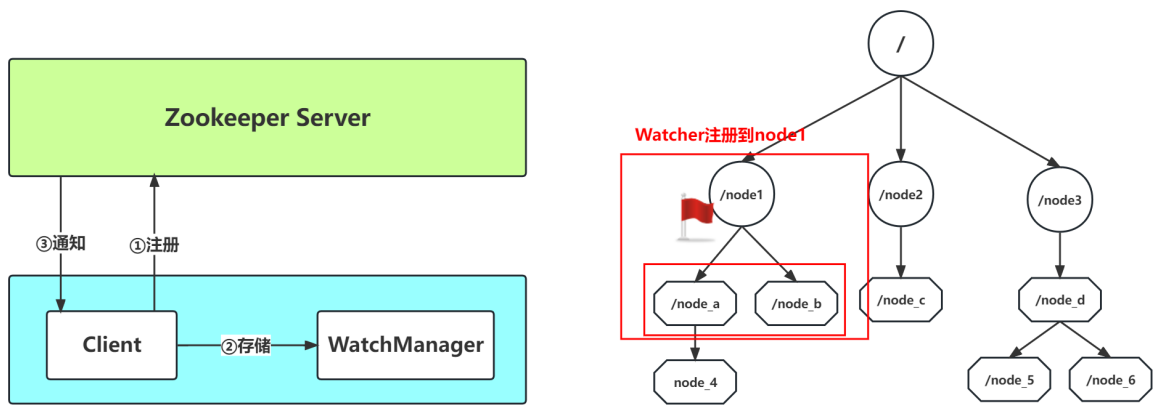
3.2 监听原理详解

Zookeeper观察者Watcher由三个部分组成，涉及消息通信及数据存储。

- 客户端Client
- 客户端监听管理器WatchManager
- Zookeeper服务器Server

监听过程：

1. Client向Server注册监听事件
2. 注册同时会将观察者Watcher存储到客户端的WatchManager里
3. Server触发监听事件后会主动向Client发送消息通知【Push】
4. Client线程从WatchManager中回调Watcher执行具体业务功能，拉取节点中数据【Pull】



看到小红旗了吧？

加入的小红旗是一个watcher，当小红旗被创建并注册到node1节点后，就会监听【node1 + node_a + node_b】或【node_a + node_b】。这里两种情况是因为在创建watcher注册时会有多种途径。但watcher不能监听到孙子辈节点。

注意：watcher设置后一旦触发一次后就会失效，如果要想一直监听，需要在process回调函数里重新注册相同的watcher。

Apache Curator框架的监听实现：

Curator对watcher机制做了优化，Curator引入了Cache的概念用来实现对Zookeeper服务器端进行事件监听。Cache是Curator对事件监听的包装，其对事件的监听可以近似看做是一个本地缓存视图和远程Zookeeper视图的对比过程。而且Curator会自动的再次监听，我们就不需要自己手动的重复监听了。

Curator客户端的Cache共有三种模式：

- 模式1-监听某节点内容变化NodeCache
- 模式2-监听某节点的子节点变化，注意不关注节点本身和孙子节点PathChildrenCache
- 模式3-监听整个树变化TreeCache

下面我们分别对三种Cache进行介绍

3.3 监听三种模式

3.3.1 模式1-监听节点

NodeCache监听指定节点的数据变化。当监听指定节点的数据发生变化时，回调对应方法。

实现代码：

```

1 public class Demo02Watch {
2
3     CuratorFramework client;
4     @Before
5     public void initClient() {
6         //初始化Zookeeper的客户端对象client
7         String connectString = "127.0.0.1:2181";
8         RetryPolicy retryPolicy = new ExponentialBackoffRetry(3000, 100);
9         int sessionTimeoutMs = 60 * 1000; //当前客户端会话超时时间
10        int connectionTimeoutMs = 15 * 1000; //连接超时时间

```

```

11     client = CuratorFrameworkFactory
12         .newClient(connectString, sessionTimeoutMs,
connectionTimeoutMs, retryPolicy);
13     }
14     @After
15     public void destroy(){
16         client.close();
17     }
18
19     /**
20      * 1.监听节点数据变化
21      */
22     @Test
23     public void listenNode() throws Exception {
24         //1.启动客户端
25         client.start();
26         System.out.println("连接zookeeper成功~~~~~");
27         //2.创建节点监听对象NodeCache : 设置监听节点、监听回调方法
28         NodeCache nodeCache = new NodeCache(client, "/hero");
29         //设置监听节点
30         ChildData currentData = nodeCache.getCurrentData();
31         System.out.println("当前节点数据 = " + currentData);
32         //开启监听
33         nodeCache.start(true);
34         //设置监听回调方法
35         nodeCache.getListenable().addListener(new NodeCacheListener() {
36             /**
37              * 如果节点数据有变化，回调当前方法
38              */
39             @Override
40             public void nodeChanged() throws Exception {
41                 //获取当前节点的数据
42                 ChildData currentData = nodeCache.getCurrentData();
43                 //获取最新节点名称
44                 String path = currentData.getPath();
45                 System.out.println("节点名称 = " + path);
46                 //获取最新节点数据
47                 byte[] currentDataByte = currentData.getData();
48                 System.out.println("修改后节点数据" + new
String(currentDataByte));
49                 System.out.println("----->>");
50             }
51         });
52         //从输入流中读取数据的下一个字节。
53         //阻塞主线程
54         //System.in.read();
55         Thread.sleep(100000);
56     }
57 }

```

命令行修改节点

```

[zk: localhost:2181(CONNECTED) 15] create /hero
Created /hero
[zk: localhost:2181(CONNECTED) 16] set /hero 'abc'
[zk: localhost:2181(CONNECTED) 17] set /hero 'abc'
[zk: localhost:2181(CONNECTED) 18] set /hero 'bcd'

```

观察控制台

```
节点名称 = /hero
修改后节点数据abc
----->>
DEBUG rg.apache.zookeeper.ClientCnxn - Got notification sessionId:0x100010098b8000e
DEBUG rg.apache.zookeeper.ClientCnxn - Got WatchedEvent state:SyncConnected type:NodeDataChanged path:/hero
DEBUG rg.apache.zookeeper.ClientCnxn - Reading reply sessionId:0x100010098b8000e, packet:: clientPath:/hero
TRACE ator.utils.DefaultTracerDriver - Trace: ExistsBuilderImpl-Background - 1 ms
DEBUG rg.apache.zookeeper.ClientCnxn - Reading reply sessionId:0x100010098b8000e, packet:: clientPath:/hero
TRACE ator.utils.DefaultTracerDriver - Trace: GetDataBuilderImpl-Background - 1 ms
节点名称 = /hero
修改后节点数据bcd
```

3.3.2 模式2-监听节点的子节点

PathChildrenCache是用来监听指定节点的子节点变化情况【新增、修改、删除】。

三种启动模式：

1. StartMode.normal 普通模式：在后台初始化值启动，会发布现有节点和新节点的事件
2. StartMode.build_initial_cache构建初始化缓存模式：在前台初始化值，先调用rebuild再执行start，不会发布现有节点和新节点事件
3. StartMode.post_initialized_event 初始化之后调用事件模式：在后台初始化值启动，缓存完毕后，会发布Type.INITIALIZED事件

七种回调事件类型：

1. Type.CONNECTION_RECONNECTED 重新连接
2. Type.connection_lost 连接丢失
3. Type.connection_suspended 连接暂停
4. Type.INITIALIZED 初始化
5. Type.CHILD_REMOVED 子节点移除
6. Type.CHILD_ADDED 子节点添加
7. Type.CHILD_UPDATED 子节点修改

实现代码：

```
1  /**
2   * 2. 监听当前节点的子节点变化，不含节点数据
3   */
4  @Test
5  public void listenSubNode() throws Exception {
6      //1. 启动客户端
7      client.start();
8      System.out.println("连接Zookeeper成功~~~~~");
9      //2. 创建节点监听对象PathChildrenCache :
10     //参数3，是否缓存数据
11     PathChildrenCache childrenCache = new PathChildrenCache(client,
12     "/hero", true);
13     // 开启监听
14     childrenCache.start(PathChildrenCache.StartMode.POST_INITIALIZED_EVENT);
15     // 设置监听回调方法
```

```

15         childrenCache.getListenable().addListener((client, event) -> {
16             //获取修改的数据
17             byte[] bytes = event.getData().getData();
18             System.out.println("节点内数据 = " + new String(bytes));
19             //获取被修改的子节点
20             System.out.println("节点名称 = " + event.getData().getPath());
21             //获取事件类型
22             //PathChildrenCacheEvent.Type.CONNECTION_RECONNECTED 重新连接
23             //PathChildrenCacheEvent.Type.connection_lost 连接丢失
24             //PathChildrenCacheEvent.Type.connection_suspended 连接暂停
25             //PathChildrenCacheEvent.Type.INITIALIZED 初始化
26             //PathChildrenCacheEvent.Type.CHILD_REMOVED 子节点移除
27             //PathChildrenCacheEvent.Type.CHILD_ADDED 子节点添加
28             //PathChildrenCacheEvent.Type.CHILD_UPDATED 子节点修改
29             PathChildrenCacheEvent.Type type = event.getType();
30             System.out.println("事件触发类型 = " + type);
31             System.out.println("----->>");
32         });
33         //3.阻塞程序
34         System.in.read();
35     }

```

3.3.3 模式3-监听所有节点【模式1+模式2】

TreeCache模式像上面两种模式的合体，TreeCache既能够监听自身节点的数据变化、也能够监听子节点的变化【新增、修改、删除】。

实现代码：

```

1  /**
2   * 3. 树形监听所有下级节点变化【模式1+模式2】，含节点数据变更
3   */
4  @Test
5  public void treeCache() throws Exception {
6      //1.启动客户端
7      client.start();
8      System.out.println("连接Zookeeper成功~~~~~");
9      //2.创建节点监听对象TreeCache :
10     TreeCache treeCache = new TreeCache(client, "/hello");
11     //启动缓存
12     treeCache.start();
13     //添加监听回调方法
14     treeCache.getListenable().addListener(new TreeCacheListener() {
15         @Override
16         public void childEvent(CuratorFramework client, TreeCacheEvent
17             event) throws Exception {
18             //获取修改的数据
19             byte[] bytes = event.getData().getData();
20             System.out.println("节点内数据 = " + new String(bytes));
21             //获取被修改的子节点
22             System.out.println("节点名称 = " + event.getData().getPath());
23             //获取事件类型

```



```

23      //TreeCacheEvent.Type.CONNECTION_RECONNECTED 重新连接
24      //TreeCacheEvent.Type.connection_lost 连接丢失
25      //TreeCacheEvent.Type.connection_suspended 连接暂停
26      //TreeCacheEvent.Type.INITIALIZED 初始化
27      //TreeCacheEvent.Type.NODE_REMOVED 子节点移除
28      //TreeCacheEvent.Type.NODE_ADDED 子节点添加
29      //TreeCacheEvent.Type.NODE_UPDATED 子节点修改
30      TreeCacheEvent.Type type = event.getType();
31      System.out.println("事件触发类型 = " + type);
32      System.out.println("----->>");
33      }
34  });
35  //3.阻塞程序
36  System.in.read();
37  }

```

4. 集群搭建

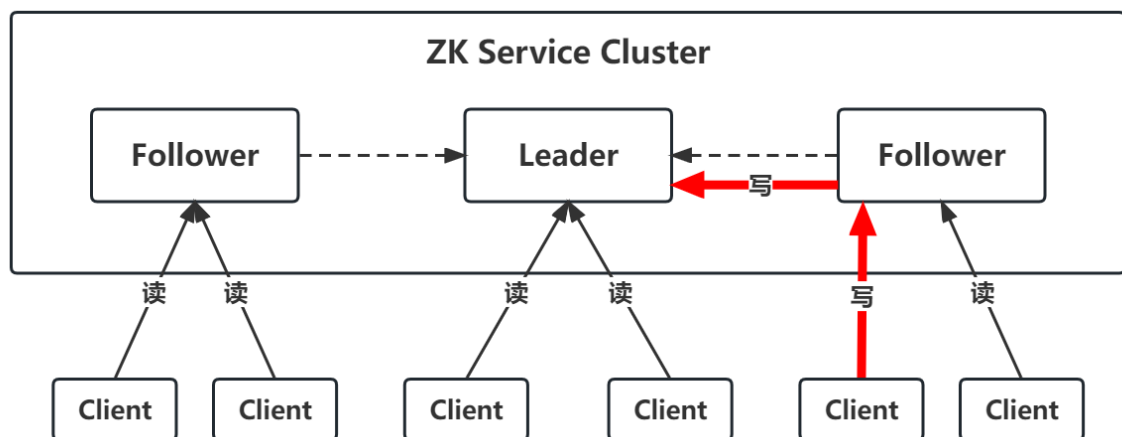
4.1 简介

本Zookeeper 集群搭建指的是 Zookeeper 分布式模式安装。由 $2n+1$ 台 servers 组成。

Zookeeper集群搭建有两种方式：

- **单机分布式集群搭建：**
 - 在一台机器上启动多个Zookeeper，在启动每个Zookeeper时分别使用不同的配置文件 zoo.cfg来启动，每个配置文件使用不同的配置参数(clientPort端口号、dataDir数据目录。
 - 在zoo.cfg中配置多个server.id，其中ip都是当前机器，而端口各不相同。
- **多机分布式集群搭建：**
 - 多台机器各自配置zoo.cfg文件，将各自互相加入服务器列表，每个节点占用一台机器

4.2 集群架构图



Leader主节点是集群工作的核心：

- 事务请求的唯一调度和处理者（写操作）对于 create, setData, delete 等有写操作的请求，则需要统一转发给Leader处理，主节点需要决定编号、执行操作，这个过程称为一个事务。
- 保证集群事务处理的顺序性

Follower从节点

- 处理客户端非事务（读操作）请求，转发事务请求给 Leader
- 参与集群 Leader 选举投票 2n-1台可以做集群投票

Observer观察者节点

- 针对访问量比较大的 Zookeeper 集群，还可新增观察者角色。观察 Zookeeper 集群的最新状态变化并将这些状态同步过来，其对于非事务请求可以进行独立处理，对于事务请求，则会转发给 Leader服务器进行处理。
- 不会参与任何形式的投票只提供非事务服务，通常用于在不影响集群事务处理能力的前提下提升集群的非事务处理能力。

4.3 搭建过程

准备三台服务器并安装好DK

4.3.3 上传Zookeeper到虚拟机并解压

上传安装包到虚拟机

```
1 ## 进入上传文件路径，解压安装包
2 tar -vxf /root/apache-zookeeper-3.6.2-bin.tar.gz
3 ## 创建zookeeper文件夹
4 mkdir /usr/local/hero/apache-zookeeper-3.6.2
5 ## 复制到指定文件夹下
6 mv /root/apache-zookeeper-3.6.2-bin/* /usr/local/hero/apache-zookeeper-3.6.2/
```

4.3.4 配置Zookeeper

- 创建data目录

```
1 ## 进入Zookeeper目录
2 cd /usr/local/hero/apache-zookeeper-3.6.2/
3 ## 创建data目录
4 mkdir data
```

- 修改conf/zoo.cfg

```
1 ## 进入conf目录
2 cd /usr/local/hero/apache-zookeeper-3.6.2/conf
3 ## 复制zoo_sample.cfg，文件名为zoo.cfg
4 cp zoo_sample.cfg zoo.cfg
```

- 修改配置文件zoo.cfg

```

1  ## 修改配置文件
2  vim /usr/local/hero/apache-zookeeper-3.6.2/conf/zoo.cfg
3  ## 数据存储目录
4  dataDir=/usr/local/hero/apache-zookeeper-3.6.2/data
5  ## 集群配置地址
6  server.1=172.17.187.79:2182:3182
7  server.2=172.17.187.79:2183:3183
8  server.3=172.17.187.79:2184:3184

```

```

# The number of milliseconds of each tick
tickTime=2000
# The number of ticks that the initial
# synchronization phase can take
initLimit=10
# The number of ticks that can pass between
# sending a request and getting an acknowledgement
syncLimit=5
# the directory where the snapshot is stored.
# do not use /tmp for storage, /tmp here is just
# example sake.
dataDir=/usr/local/zookeeper/apache-zookeeper-3.6.2-bin/data  数据存储路径
# the port at which the clients will connect
clientPort=2181  服务端口
#autopurge.snapRetainCount=3
# Purge task interval in hours
# Set to "0" to disable auto purge feature
#autopurge.purgeInterval=1
# 集群配置地址
server.1=192.168.186.128:2182:3182
server.2=192.168.186.129:2182:3182
server.3=192.168.186.130:2182:3182  集群机器配置

```

参数详解(了解)

- 1) tickTime: 通信心跳数
Zookeeper服务器心跳时间，单位毫秒
Zookeeper使用的基本时间，服务器之间或客户端与服务器之间维持心跳的时间间隔，也就是每个tickTime时间就会发送一个心跳，时间单位为毫秒。
它用于心跳机制，并且设置最小的session超时时间为两倍心跳时间。(session的最小超时时间是2*tickTime)
- 2) initLimit: LF初始通信时限
集群中的follower跟随者服务器(F)与leader领导者服务器(L)之间初始连接时能容忍的最多心跳数(tickTime的数量)，用它来限定集群中的Zookeeper服务器连接到Leader的时限。
投票选举新leader的初始化时间
Follower在启动过程中，会从Leader同步所有最新数据，然后确定自己能够对外服务的起始状态。
Leader允许F在initLimit时间内完成这个工作。
- 3) syncLimit: LF同步通信时限
集群中Leader与Follower之间的最大响应时间单位，假如响应超过syncLimit * tickTime，Leader认为Follower死掉，从服务器列表中删除Follower。
在运行过程中，Leader负责与ZK集群中所有机器进行通信，例如通过一些心跳检测机制，来检测机器的存活状态。
如果L发出心跳包在syncLimit之后，还没有从F那收到响应，那么就认为这个F已经不在线了。
- 4) dataDir: 数据文件目录+数据持久化路径
保存内存数据库快照信息的位置，如果没有其他说明，更新的事务日志也保存到数据库。
- 5) clientPort: 客户端连接端口
监听客户端连接的端口

```

24 6) 集群中服务的列表
25     server.1=172.17.187.79:2182:3182
26     server.2=172.17.187.79:2183:3183
27     server.3=172.17.187.79:2184:3184
28
29 server.* 后面的数据对应myid中的数字
30 ip地址对应每台虚拟机的ip地址
31 2182: 表示的是这个服务器与集群中的 Leader 服务器交换信息的端口
32 3182: 表示的是万一集群中的 Leader 服务器挂了, 需要一个端口来重新进行选举, 选出一个新的
    Leader, 而这个端口就是用来执行选举时服务器相互通信的端口

```

- 在data目录创建创建myid

```

1  ## 修改myid文件,分别设置为1,2,3
2  vim /usr/local/hero/apache-zookeeper-3.6.2/data/myid
3  vim /usr/local/hero/node01/data/myid
4  vim /usr/local/hero/node02/data/myid
5  vim /usr/local/hero/node03/data/myid

```

```

1  ## 修改集群配置
2  server.1=172.17.187.79:2182:3182
3  server.2=172.17.187.79:2183:3183
4  server.3=172.17.187.79:2184:3184
5
6  ## 修改服务编号【按照服务节点修改1,2,3,4,5】
7  touch /usr/local/hero/node01/data/myid
8  touch /usr/local/hero/node02/data/myid
9  touch /usr/local/hero/node03/data/myid
10 sudo tee /usr/local/hero/node01/data/myid <<- 'EOF'
11 1
12 EOF
13 sudo tee /usr/local/hero/node02/data/myid <<- 'EOF'
14 2
15 EOF
16 sudo tee /usr/local/hero/node03/data/myid <<- 'EOF'
17 3
18 EOF

```

4.3.5 启动Zookeeper

```

1  ## 停止zk
2  /usr/local/hero/node01/bin/zkServer.sh stop
3  /usr/local/hero/node02/bin/zkServer.sh stop
4  /usr/local/hero/node03/bin/zkServer.sh stop
5  ## 启动zk
6  /usr/local/hero/node01/bin/zkServer.sh start
7  /usr/local/hero/node02/bin/zkServer.sh start
8  /usr/local/hero/node03/bin/zkServer.sh start

```

4.3.6 查看Zookeeper状态

```
1 ## 查看zk状态
2 /usr/local/hero/node01/bin/zkServer.sh status
3 /usr/local/hero/node02/bin/zkServer.sh status
4 /usr/local/hero/node03/bin/zkServer.sh status
```

5. 案例：手写分布式锁

在单体的应用开发场景中，涉及并发同步的时候，大家往往采用synchronized或者Lock的方式来解决多线程间的同步问题。但在分布式集群工作的开发场景中，那么就需要一种更加高级的锁机制，来处理跨JVM进程之间的数据同步问题，这就是分布式锁。

5.1 回顾公平锁和可重入锁

什么是可重入的公平锁呢？

5.2 Java锁分类

01-按上锁方式划分

- ① 隐式锁：synchronized，不需要显示加锁和解锁
- ② 显式锁：JUC包中提供的锁，需要显示加锁和解锁

02-按特性划分

➤ 2.1 悲观锁/乐观锁：按照线程在使用共享资源时，要不要锁住同步资源，划分为悲观锁和乐观锁

- 悲观锁：JUC锁，synchronized
- 乐观锁：CAS，关系型数据库的版本号机制

➤ 2.2 重入锁/不可重入锁：按照同一个线程是否可以重复获取同一把锁，划分为重入锁和不可重入锁

- 重入锁：ReentrantLock、synchronized
- 不可重入锁：不可重入锁，与可重入锁相反，线程获取锁之后不可重复获取锁，重复获取会发生死锁

5.2 Java锁分类

02-按特性划分

➤ 2.3 公平锁/非公平锁：按照多个线程竞争同一锁时需不需要排队，能不能插队，划分为公平锁和非公平锁。

- 公平锁：new ReentrantLock(true)多个线程按照申请锁的顺序获取锁
- 非公平锁：new ReentrantLock(false)多个线程获取锁的顺序不是按照申请锁的顺序(可以插队) synchronized

5.2 实现原理

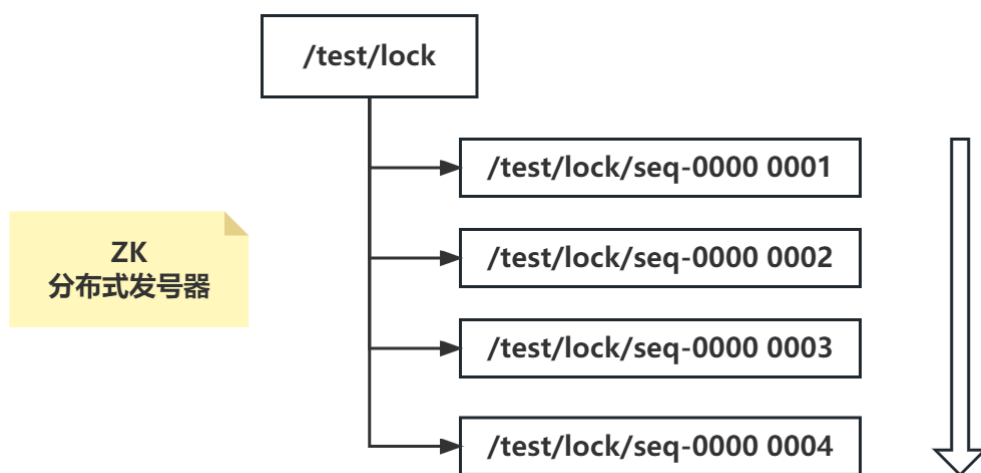
理解了公平可重入锁概念后，接下来分析ZK如何实现分布式的锁，公平的且是可重入的锁

5.2.1 如何实现线程排队？

zNode节点可以是一个发号器（排队）。

在一个节点下面创建临时顺序节点（EPHEMERAL_SEQUENTIAL类型），新的子节点会加上次序编号，而这个次序编号是上一个生成的次序编号加一。

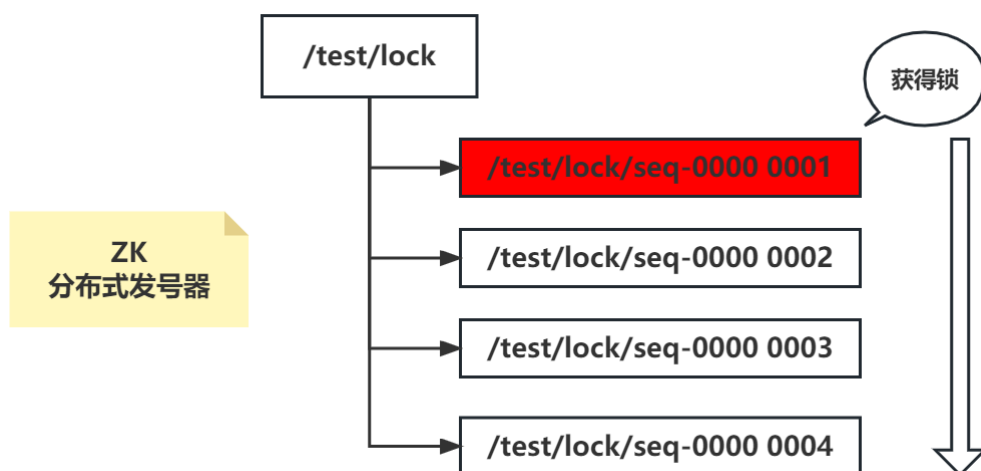
举个栗子：有一个用于发号的节点“/test/lock”为父亲节点，可以在这个父节点下面创建相同前缀的临时顺序子节点，假定相同的前缀为“/test/lock/seq-”。第一个创建的子节点基本上应该为/test/lock/seq-0000000000，下一个节点则为/test/lock/seq-0000000001，依次类推，如图所示。



5.2.2 如何确保公平获取锁？

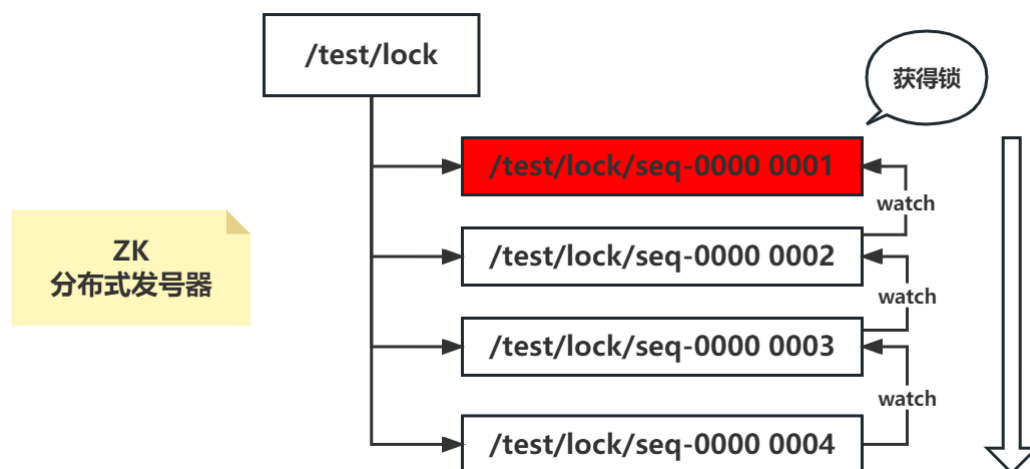
节点的递增有序性，可以确保锁的公平。

ZK实现分布式锁，首先需要创建一个父节点（PERSISTENT类型），然后每个要获得锁的线程，都在这个节点下创建个临时顺序节点。由于ZK节点是按照创建的次序，依次递增。为了确保公平可以规定：**编号最小的节点表示获得锁**。所以，每个线程在尝试占用锁之前，首先判断自己是排号是不是当前最小，如果是则获得锁。



5.2.3 如何实现线程等待唤醒的机制？

节点监听机制可以实现线程获取锁的等待和唤醒的传递有序而且高效。



每个线程抢占锁之前，先尝试创建自己的ZNode。同样，释放锁的时，需要删除创建的Znode。创建成功后，如果不是排号最小的节点，就处于等待通知状态。

等待谁的通知呢？等待序号排在前一个的Znode 通知。前一个Znode删除时会触发Znode通知事件，当前节点能监听到删除事件，然后尝试获取锁，如果没有获取到，则如此向下继续排队，第二个通知第三个、第三个通知第四个，击鼓传花依次向后。**ZK节点监听机制能够非常完美地实现这种效果。**

具体实现思路：每个等通知的Znode节点，只需要监听（listen）或者监视（watch）排号在自己前面那个节点，这样就能收到上个节点删除事件。只要上个节点删除，就判断自己是不是序号最小的节点，如果是获取锁，不是继续排队等待。

如果因为网络中断，线程阻塞了排队怎么办？

无需担心，ZK可以解决这个问题。如果由于网络异常或者其他原因，集群中占用锁的客户端失联时，这个临时节点会自行释放，线程也就从排队中消失了。排在它后面的那个节点，也能收到删除事件，从而继续排队或者获得锁。正是由于这个原因，在创建取号节点的时候，需创建**临时节点**。

如何避免羊群效应？

所谓羊群效应就是一个节点挂掉，所有节点都去监听然后做出反应，这样会短时给服务器带来巨大压力。上述有序节点排队，下个节点监听上个节点的机制，当一个节点挂掉，只有它后面的那一个节点才做出反应，可以很好的避免羊群效应。

5.2.4 实现思路

1. 一把分布式锁通常使用一个Znode节点表示
 - 如果锁对应的Znode节点不存在，首先创建Znode节点
 - 这里假设为“/test/lock”代表了一把需要创建的分布式锁
2. 抢占锁的所有客户端，使用锁的Znode节点的子节点列表来表示

- 如果某个客户端需要占用锁，则在“/test/lock”下创建一个临时有序的子节点
 - 所有临时有序子节点，尽量共用一个有意义的子节点前缀，比如前缀为“/test/lock/seq-”
 - 第一次抢锁对应的子节点为“/test/lock/seq-000000000”
 - 第二次抢锁对应的子节点为“/test/lock/seq-000000001”，以此类推
3. 如果判定客户端是否占有锁呢？两种情况
- 情况01-自己创建的子节点，判断**是否为当前子节点列表中序号最小的子节点**
 - 如果是，加锁成功；
 - 如果不是，则监听前一个Znode子节点变更消息，等待前一个节点释放锁
 - 情况02-一旦队列中的后面的节点，获得前一个子节点变更通知，则判断**是否为当前子节点列表中序号最小的子节点**
 - 如果是，加锁成功
 - 如果不是，则持续监听，一直到获得锁
4. 获取锁后，开始处理业务流程，处理完成后删除对应子节点，完成释放锁的工作，以方便后继节点能获得分布式锁

5.3 加锁和释放锁的实现

原理说清楚了，接下来咱们就基于ZooKeeper实现下分布式锁。

首先定义锁接口Lock，锁动作就两个加锁和解锁，所以定义一个加锁方法和一个解锁方法。代码如下：

```

1  public interface Lock {
2      /**
3       * 加锁方法
4       * @return 是否成功加锁
5       */
6      boolean lock() throws Exception;
7
8      /**
9       * 解锁方法
10      * @return 是否成功解锁
11      */
12      boolean unlock();
13  }
```

5.3.1 加锁

Lock接口中加锁的方法是lock()。lock()方法的大致流程是：首先尝试着去加锁，如果加锁失败就去等待，然后再重复。

1) lock方法实现代码

lock()方法加锁的实现代码如下：

```

1  @Slf4j
2  public class DLHeroLock implements Lock {
3
4      private static final String ZK_PATH = "/DLHero/lock";//DLHeroLock的节点链
      接
5      private static final String LOCK_PREFIX = ZK_PATH + "/"//锁前缀
```



```

6
7     private static final long WAIT_TIME = 1000; //锁排队超时时间
8
9
10    private String locked_path = null; //当前节点，举个栗子
    子: /DLHero/lock/0000000015
11    private String prior_path = null; //当前节点的上一个节点，举个栗子
    子: /DLHero/lock/0000000014
12    private String locked_short_path = null; //节点短路径，举个栗子: 0000000015
13    final AtomicInteger lockCount = new AtomicInteger(0); //重入锁计数器
14    private CuratorFramework client = null; //zk客户端
15
16    private Thread thread; //本地线程
17
18    public DLHeroLock() {
19        ZKClient.instance.init();
20        if (!ZKClient.instance.isNodeExist(ZK_PATH)) {
21            ZKClient.instance.createNode(ZK_PATH, null);
22        }
23        client = ZKClient.instance.getClient();
24    }
25
26    @Override
27    public boolean lock() {
28        //可重入，确保同一线程，可以重复加锁
29        synchronized (this) {
30            if (lockCount.get() == 0) {
31                thread = Thread.currentThread();
32                lockCount.incrementAndGet();
33            } else {
34                if (!thread.equals(Thread.currentThread())) {
35                    return false;
36                }
37                lockCount.incrementAndGet();
38                return true;
39            }
40        }
41        try {
42            boolean locked = false;
43            //首先尝试着去加锁
44            locked = tryLock();
45            if (locked) {
46                return true;
47            }
48            //如果加锁失败就去等待
49            while (!locked) {
50                await();
51                //获取等待的子节点列表
52                List<String> waiters = getwaiters();
53                //判断，是否加锁成功
54                if (checkLocked(waiters)) {
55                    locked = true;
56                }
57            }
58            return true;

```

```

59         } catch (Exception e) {
60             e.printStackTrace();
61             unlock();//释放锁并减少计数
62         }
63         return false;
64     }
65 }

```

2) tryLock尝试加锁

尝试加锁的tryLock方法是关键，做了两件重要的事情：

1. 创建临时顺序节点，并且保存自己的节点路径
2. 判断是否是第一个节点
 - 如果是第一个，则加锁成功
 - 如果不是，就找到前一个Znode节点，并且保存其路径到prior_path

尝试加锁的tryLock方法实现如下：

```

1      /**
2       * 尝试加锁
3       * @return 是否加锁成功
4       * @throws Exception 异常
5       */
6      private boolean tryLock() throws Exception {
7          //创建临时Znode
8          locked_path = ZKClient.instance.createEphemeralSeqNode(LOCK_PREFIX);
9          //然后获取所有节点
10         List<String> waiters = getwaiters();
11         if (null == locked_path) {
12             throw new Exception("zk error");
13         }
14         //取得加锁的排队编号
15         locked_short_path = getShorPath(locked_path);
16
17         //获取等待的子节点列表，判断自己是否第一个
18         if (checkLocked(waiters)) {
19             return true;
20         }
21         // 判断自己排第几个
22         int index = Collections.binarySearch(waiters, locked_short_path);
23         if (index < 0) { // 网络抖动，获取到的子节点列表里可能已经没有自己了
24             throw new Exception("节点没有找到: " + locked_short_path);
25         }
26         //如果自己没有获得锁，则要监听前一个节点
27         prior_path = ZK_PATH + "/" + waiters.get(index - 1);
28
29         return false;
30     }

```

创建临时顺序节点后，其完整路径存放在locked_path成员中；另外还截取了一个后缀路径，放在locked_short_path成员中，后缀路径是一个短路径，只有完整路径的最后一层。

为什么要单独保存短路径呢？

因为，在获取的远程子节点列表中的其他路径返回结果时，返回的都是短路径，都只有最后一层路径。所以为了方便后续进行比较，也把自己的短路径保存下来。

创建了自己的临时节点后，调用checkLocked方法，判断是否是锁定成功。如果锁定成功，则返回true；如果自己没有获得锁，则要监听前一个节点，此时需要找出前一个节点的路径，并保存在prior_path成员中，供后面的await()等待方法去监听使用。在进入await()等待方法的介绍前，先说下checkLocked 锁定判断方法。

3) checkLocked检查是否持有锁

在checkLocked()方法中，判断是否可以持有锁。判断规则很简单：当前创建的节点，是否在上一步获取到的子节点列表的第一个位置：

- 如果是，说明可以持有锁，返回true，表示加锁成功；
- 如果不是，说明有其他线程早已先持有了锁，返回false。

checkLocked()方法代码如下：

```
1      //检查是否持有锁
2      private boolean checkLocked(List<String> waiters) {
3          //节点按照编号，升序排列
4          Collections.sort(waiters);
5          // 如果是第一个，代表自己已经获得了锁
6          if (locked_short_path.equals(waiters.get(0))) {
7              log.info("成功的获取分布式锁,节点为{}", locked_short_path);
8              return true;
9          }
10         return false;
11     }
```

checkLocked方法比较简单，将参与排队的所有子节点列表，从小到大根据节点名称进行排序。排序主要依靠节点的编号，也就是后Znode路径的10位数字，因为前缀都是一样的。排序之后，做判断，如果自己的locked_short_path编号位置排在第一个，如果是，则代表自己已经获得了锁。如果不是，则会返回false。

如果checkLocked()为false，外层的调用方法，一般来说会执行await()等待方法，执行夺锁失败以后的等待逻辑。

4) await监听前一个节点释放锁

await()也很简单，就是监听前一个ZNode节点（prior_path成员）的删除事件，代码如下：

```
1      private void await() throws Exception {
2          if (null == prior_path) {
3              throw new Exception("prior_path error");
4          }
5          final CountdownLatch latch = new CountdownLatch(1); //倒数门闩
6          //订阅比自己次小顺序节点的删除事件
7          watcher w = new watcher() {
8              @Override
9              public void process(WatchedEvent watchedEvent) {
```

```

10         System.out.println("监听到的变化 watchedEvent = " +
watchedEvent);
11         log.info("[watchedEvent] 节点删除");
12         latch.countDown();
13     }
14 };
15
16     client.getData().usingWatcher(w).forPath(prior_path);
17
18     latch.await(WAIT_TIME, TimeUnit.SECONDS);
19 }

```

首先添加一个Watcher监听，而监听的节点，正是前面所保存在prior_path成员的前一个节点的路径。这里，仅仅去监听自己前一个节点的变动，而不是其他节点的变动，提升效率。完成监听之后，调用latch.await()，线程进入等待状态，一直到线程被监听回调代码中的latch.countDown()所唤醒或者等待超时。

上面的代码中，监听前一个节点的删除，可以使用两种监听方式：

- Watcher 订阅
- TreeCache 订阅

两种方式的效果，都差不多。但是这里的删除事件，只需要监听一次即可，不需要反复监听，所以使用的是Watcher 一次性订阅。而TreeCache 订阅的代码在源码工程中已经被注释，仅供参考。

一旦前一个节点prior_path节点被删除，那么就将线程从等待状态唤醒，重新一轮的锁的争夺，直到获取锁，并且完成业务处理。

至此分布式Lock加锁的算法基本完成，还差可重入的逻辑实现。

5) 可重入的实现代码

什么是可重入呢？只需要保障同一个线程进入加锁的代码，可以重复加锁成功即可。

修改前面的lock方法，在前面加上可重入的判断逻辑。代码如下：

```

1     @Override
2     public boolean lock() {
3         //可重入，确保同一线程，可以重复加锁
4         synchronized (this) {
5             if (lockCount.get() == 0) {
6                 thread = Thread.currentThread();
7                 lockCount.incrementAndGet();
8             } else {
9                 if (!thread.equals(Thread.currentThread())) {
10                     return false;
11                 }
12                 lockCount.incrementAndGet();
13                 return true;
14             }
15         }
16         //....
17     }
18     @Override
19     public boolean unlock() {

```

```

20         //只有加锁的线程，能够解锁
21         if (!thread.equals(Thread.currentThread())) {
22             return false;
23         }
24         //减少可重入的计数
25         int newLockCount = lockCount.decrementAndGet();
26         //计数不能小于0
27         if (newLockCount < 0) {
28             throw new IllegalMonitorStateException("重入锁计数不可为负数: " +
locked_path);
29         }
30         //如果计数不为0，直接返回
31         if (newLockCount != 0) {
32             return true;
33         }
34         //....
35     }

```

为了变成可重入，在代码中增加了一个加锁的计数器lockCount，计算重复加锁的次数。如果是同一个线程加锁，只需要增加次数，直接返回，表示加锁成功。到这里lock()方法已经介绍完成，接下来，就是去释放锁

5.3.2 释放锁

Lock接口中的unlock()方法，表示释放锁，释放锁主要有两个工作：

- 减少重入锁的计数，如果最终的值不是0，直接返回，表示成功的释放了一次；
- 如果计数器为0，移除Watchers监听器，并且删除创建的Znode临时节点。

unlock方法的代码如下：

```

1     /**
2      * 释放锁
3      * @return 是否成功释放锁
4      */
5     @Override
6     public boolean unlock() {
7         //只有加锁的线程，能够解锁
8         if (!thread.equals(Thread.currentThread())) {
9             return false;
10        }
11        //减少可重入的计数
12        int newLockCount = lockCount.decrementAndGet();
13        //计数不能小于0
14        if (newLockCount < 0) {
15            throw new IllegalMonitorStateException("Lock count has gone
negative for lock: " + locked_path);
16        }
17        //如果计数不为0，直接返回
18        if (newLockCount != 0) {
19            return true;
20        }
21        //删除临时节点
22        try {

```

```

23         if (zkClient.instance.isNodeExist(locked_path)) {
24             client.delete().forPath(locked_path);
25         }
26     } catch (Exception e) {
27         e.printStackTrace();
28         return false;
29     }
30     return true;
31 }

```

这里，为了尽量保证线程安全，可重入计数器的类型，使用的不是int类型，而是原子类AtomicInteger。

5.4 分布式锁测试用例

写一个用例，测试一下DLHeroLockTest的使用，代码如下：

```

1  @Slf4j
2  public class DLHeroLockTest {
3
4      int count = 0; //共享变量
5
6      @Test
7      public void testLock() throws InterruptedException {
8          for (int i = 0; i < 10; i++) {
9              FutureTaskScheduler.add(() -> {
10                 //创建锁
11                 DLHeroLock lock = new DLHeroLock();
12                 lock.lock(); //加锁
13
14                 //每条线程，执行共享变量的10次累加
15                 for (int j = 0; j < 10000; j++) {
16                     count++;
17                 }
18                 try {
19                     Thread.sleep(1000);
20                 } catch (InterruptedException e) {
21                     e.printStackTrace();
22                 }
23                 log.info("count ===== " + count);
24                 //释放锁
25                 lock.unlock();
26             });
27         }
28         Thread.sleep(Integer.MAX_VALUE);
29     }
30
31 }

```

以上代码是10个并发任务，每个任务累加10次，执行以上用例，会发现结果会是预期的和100，如果不使用锁，结果可就不是100了，因为上面的count是一个普通的变量，不是线程安全的，在单进程中如此，在多进程中也是如此。

```
INFO ero.distributedlock.DLHeroLock - 成功的获取分布式锁,节点为0000000009
INFO distributedlock.DLHeroLockTest - count = 100
DEBUG rg.apache.zookeeper.ClientCnxn - Reading reply sessionId:0x1000004bfe0000b, packet:: clientPath:null
TRACE ator.utils.DefaultTracerDriver - Trace: ExistsBuilderImpl-Foreground - 1 ms
INFO .hero.distributedlock.ZKClient - 节点存在 stat is:
DEBUG rg.apache.zookeeper.ClientCnxn - Reading reply sessionId:0x1000004bfe0000b, packet:: clientPath:null
```

5.5 Curator内置的分布式锁InterProcessMutex

分布式锁Zlock自主实现主要的价值：学习一下分布式锁的原理和基础开发，仅此而已。实际的开发中，如果需要使用到分布式锁，建议直接使用Curator客户端中的各种官方实现的分布式锁，比如其中的InterProcessMutex 可重入锁。

这里提供一个简单的InterProcessMutex 可重入锁的使用实例，代码如下：

```
1      @Test
2      public void testInterProcessMutex() throws InterruptedException {
3
4          CuratorFramework client = ZKClient.instance.getClient();
5          final InterProcessMutex zkMutex = new InterProcessMutex(client,
6              "/mutex");
7
8          for (int i = 0; i < 10; i++) {
9              FutureTaskScheduler.add(() -> {
10
11                  try {
12                      //获取互斥锁
13                      zkMutex.acquire();
14
15                      for (int j = 0; j < 10000; j++) {
16                          count++; //共享变量累加
17                      }
18                      try {
19                          Thread.sleep(1000);
20                      } catch (InterruptedException e) {
21                          e.printStackTrace();
22                      }
23                      log.info("count ===== " + count);
24                      zkMutex.release(); //释放互斥锁
25
26                  } catch (Exception e) {
27                      e.printStackTrace();
28                  }
29              });
30          }
31          Thread.sleep(Integer.MAX_VALUE);
32      }
```

5.8 ZK分布式锁的小结

优缺点：

- 优点：ZK分布式锁（如InterProcessMutex）能有效的解决分布式锁问题，不可重入问题，使用起来比较简单
- 缺点：ZK实现的分布式锁性能并不太好

为什么ZK分布式锁性能不高呢？

- 因为每次在创建锁和释放锁的过程中，都需要动态创建、销毁瞬时节点来实现锁功能。
- 众所周知 ZK中创建和删除节点只能通过Leader服务器来执行，然后Leader服务器还需要将数据同步到所有的Follower机器上，这样频繁的网络通信，性能的短板非常突出。总之在高性能、高并发的场景下不建议使用ZK分布式锁。
- 由于ZooKeeper高可用的特性，所以在大部分非高并发场景中还是推荐使用ZK的分布式锁。

目前分布式锁实现方案中比较成熟的方案有两种：

- 基于Redis分布式锁，适用于并发量很大、性能要求很高的、而可靠性问题可以通过其他方案去弥补的场景。
- 基于ZK分布式锁，适用于高可靠（高可用）而并发量不是太大的场景；

在分布式系统中ZooKeeper是一个重要的协调工具。主流的分布式协调中间件，也不仅仅只有Zookeeper，还有非常著名的Etcd中间件。但是从学习的层面来说，二者之间的功能设计、核心原理都是差不多的，掌握了Zookeeper，Etcd上手使用也是很容易的。